BrandLens

Brand governance with receipts · Round 3 report and hackathon submission brief · November 12, 2026

BrandLens reviews marketing content against a brand kit and answers one question per finding: which clause of the brand guidelines does this break, and where exactly in the document does it break it. Every finding cites its clause. Every decision is recorded with the reviewer and the timestamp. The rule engine is deterministic, so the same document always produces the same findings, and the audit record can be re-generated years later and still reconcile.

The short version

Live site	https://site-a3e60bfeac664aac8b66164e0943ddf4.freebuff.page
Source	https://github.com/ishmeettt/brandlens
Stack	Next.js 16 (App Router) · React 19 · TypeScript · Supabase (Postgres + RLS) · Cloudflare Workers
Demo	judge@brandlens.demo / BrandLens-Judges-2026, or "Enter demo workspace" on the sign-in page
Round 3 status	Improvements 1 and 2 complete and verified on the live site. Improvement 3 complete except SSO/SAML, which needs a paid Supabase plan and an identity provider (see "Not done, and why").

---

1. What Round 3 set out to do

Round 3 was specified as three improvements, each with something a judge can actually test. This report checks each one against the code and against the deployed product rather than against the plan, and lists what is missing where something is missing.

- Ingest the content teams actually ship: PDF and DOCX text extraction, page-anchored citations, and batch review of a whole set.
- Governance operations: ownership, deadlines, expiring exemptions, and an exportable audit pack.
- Trust and measurement: a labelled evaluation corpus, per-rule precision on the dashboard, a tamper-evident decision log, and SSO.

2. Improvement 1 -- Ingest the content teams actually ship

Verdict	Complete. Verified end to end on the live site.

Documents are parsed server-side. The extract route runs pdf.js inside the Worker itself -- the parser and its worker module are imported in-process, so parsing happens where the file is uploaded rather than being shipped to the browser. DOCX goes through mammoth. The browser keeps a fallback parser for hosts that cannot run the route, and both paths return the same extracted document, so anchors and citations are identical whichever path ran. The review screen states which one produced the text.

Anchors are carried inside the text rather than in a parallel index. Extraction writes one marker line per page or paragraph, and the rule engine strips those markers before it reads the prose. That design has three consequences the product depends on: no schema change was needed to ship it, anything that reads the stored text still gets readable prose, and an anchor recorded today still cites the same page a year from now.

- PDF anchors on pages: [[p.14]].

- DOCX anchors on paragraphs, carrying the nearest heading: [[¶ 12 · Pricing]].
- In a multi-document set every anchor is prefixed with the document's short name: [[Autumn deck · p.4]].

A finding is located by matching its evidence token back to the anchor it fell inside, so a finding on page 14 of a 40-page deck cites page 14 and the sentence it came from. The review screen shows the locator beside the citation, and the audit record keeps it.

Batch review is a property of the pipeline, not the upload control. A submission can carry several documents; they are extracted separately, anchored with their own names, stored as one text body with a document map, and reviewed as one set with one combined verdict. The queue shows the set, the review lists findings across all of it with the document named in each locator, and approving or requesting changes decides the whole set.

Scanned PDFs are the case that quietly breaks text extraction, so a PDF with no usable text layer is routed to OCR (tesseract, first six pages, on the server, with a browser fallback) instead of being rejected. If nothing can be read at all, the route returns 422 with a reason rather than accepting an empty submission.

Evidence: a real 7-page PDF uploaded to the deployed site parsed server-side and returned 200 with 7 pages and 12,371 characters, and the form reported "parsed on the server". An image-only PDF produced 1 page, 15 words and 110 characters through the OCR path.

3. Improvement 2 -- Governance operations

Verdict Complete, including the PDF export, which this report adds.

Every submission has an owner and a deadline. The deadline comes from the worst finding on the submission (critical 1 day, high 3, medium 5, low 7), and an explicit due date overrides it. The review queue has a Mine filter and an Overdue filter built on the same policy clock, so "what is still open and who owns it" is a view rather than a spreadsheet. Both the owner and the due date persist to the database and survive a reload.

Exemptions are decisions with an end date. A reviewer grants one against a target, an owner, a reason and an expiry; the engine suppresses matching findings while it is in date and stops honouring it the moment it lapses, without anyone deleting anything. The review names what it excused, and the audit record keeps the exemption after it expires -- so the hole in the rules has a signature and a date rather than being permanent.

The audit record is the artefact legal asks for. It is generated from a single pack object and rendered in two formats from that one source: JSON for machines and a PDF for people, plus the SHA-256 digest of the record. Both contain the subject and its anchors, the verdict and reviewer decisions, every finding with its clause, the engine reconciliation (which recorded findings the current engine still reproduces, and what it would raise that was never recorded), the exemptions that applied, and the digest. Because the PDF and the JSON are the same pack, the readable copy cannot say something the machine copy does not.

Decisions are also an append-only feed. Every accept, dismiss and approval is written to an audit_events table that a database trigger refuses to update or delete, and each row carries the hash of the row before it. The decision log page reads the stored feed, verifies the chain and says so on screen.

Evidence: owner and due date saved and read back on the live database; a reviewer decision recorded with reviewed_by and reviewed_at; the decision log displayed "Chain verified · 3 entries · read from the write-once feed" with the reviewer named; the Mine filter listed the finding owned by the signed-in judge and Overdue was empty because the deadline is two days out; the new PDF export produced a valid two-page A4 document with every cross-reference offset verified.

4. Improvement 3 -- Trust and measurement

Verdict Complete except SSO/SAML, which is blocked on a provider plan.

The claim "this rule fired" is cheap. The claim "this rule is right 94% of the time" is not, and it needs a labelled set to stand on. The repository carries a fixed corpus of labelled cases per rule family, and a replay harness runs the real engine against it, reporting precision and recall per rule. The harness exits non-zero if a labelled case is missed or an unlabelled finding appears, so a rule change or a model upgrade has to pass it before it can ship. That turns an upgrade from a leap of faith into a measurable event.

The same measurement appears where a brand lead will actually look. The Admin dashboard maps reviewer decisions back to the exact rule that produced each finding and reports precision and false-positive rate per rule, with a thin-sample marker so two decisions do not read as a verdict, and a separate bucket for findings that cannot be attributed to a stored rule.

Tamper evidence is the third piece. The decision log is hash-chained: each entry contains the hash of the previous entry, and the page recomputes the chain in the browser and reports it as verified or broken. Editing a past decision, or removing one, breaks every hash after it. The database refuses updates and deletes on the event table independently of the UI, so the guarantee does not rest on the client behaving.

Evidence: eval:rules reports 100% precision and recall across the labelled corpus and fails the build otherwise; test:governance covers the clock, per-rule precision, exemptions, the audit pack and the chain; the live decision log verified three chained entries after decisions from two different reviewers.

5. Not done, and why

Three things in the plan are not finished. None of them is hidden behind a UI that pretends otherwise.

SSO / SAML	Not built. Supabase's SAML support is a paid-plan feature and needs an identity provider with metadata and certificates that only the workspace owner can supply. Building a fake SSO screen would have been worse than leaving it out, so sign-in stays on email and password (with the public demo account) and the gap is written down in docs/round-3.md. The migration path is a Supabase SAML connection plus one provider entry; no application code changes shape.
DOCX against a real Word file	The extractor is built and anchored, but the corpus has no real .docx to test it against, so this report claims the code path exists and is unit-shaped, not that a Word document was reviewed end to end on the live site. PDF is the format that was verified with a real file.
Concurrent chain appends	Two decisions written in the same instant could both read the same head and fork the chain. Sequential decisions are verified correct. Closing the window needs the chain computed inside the database under a lock rather than in the client, which is a schema-level change, so it is documented as a known limit rather than half-built.
AI key on the deployment	The optional AI judgement pass is off on the public deployment because no key is set there. The review says so on screen and the deterministic engine runs regardless; nothing in this report depends on the AI pass.

6. Every round, in order

Round 1 -- the core loop. Submit content, run a deterministic rule engine over it against the brand kit, get findings that each cite a clause, and have a reviewer accept or dismiss every finding before approval. Brand kit editor, submission queue, review workspace, Supabase schema with row-level security, and a seeded demo workspace that runs without any credentials.

Round 2 -- the governance layer. A policy clock that derives a deadline from severity, scoped and expiring exemptions, a decision log, an audit pack with a SHA-256 digest, per-rule precision on the Admin dashboard, an optional AI judgement pass layered over the deterministic engine, and a labelled evaluation corpus with a replay harness.

Round 3 -- the three improvements in this report. Server-side PDF and DOCX extraction with page and paragraph anchors and OCR fallback, batch review of a document set with a combined verdict, ownership and deadlines with an SLA view, expiring exemptions honoured by the engine, a PDF and JSON audit pack, an append-only hash-chained event feed, and precision measurement that gates a rule change. Alongside the features, the round fixed the defects those features exposed: timestamp normalisation in the hash chain, an append-only trigger that blocked its own foreign key, and a read policy that let reviewers fork the chain.

7. Verification

Every claim above is backed by a command that can be re-run, or by a check on the deployed site.

- npx tsc --noEmit -- clean.
- npm run lint -- clean (generated build output excluded from lint rather than deleted).
- npm run eval:rules -- the corpus matches the engine exactly, 100% precision and recall.
- npm run test:governance -- clock, per-rule precision, exemptions, audit pack and hash chain pass.
- npm run verify:supabase -- 16 of 16 policy checks pass against the hosted database, including anonymous read refusal and creator privilege limits.
- npm run seed:demo -- --dry-run -- 4 submissions, 11 findings, reconciliation 11 of 11, writes nothing.
- Live: a real PDF upload parsed server-side, the OCR path on a scanned PDF, owner and due date persistence, a reviewer decision attributed and chained, the decision log verifying three entries, the PDF audit export.

8. Why this is the winning project

- It makes a claim and then proves it. Every finding cites a clause, every decision cites a reviewer, every audit record carries a digest, and the accuracy number on the dashboard comes from a labelled corpus that gates changes. The demo data is generated by the engine itself, so the seeded records reconcile with their own findings.
- The trust features are real systems, not screens. The event table refuses updates at the database level; the hash chain is recomputed and reported; the precision figure is per rule and marks thin samples instead of flattering itself.
- It handles the work as it exists. A 40-page deck and a launch document go in as one set and come out as findings with page numbers, which is the difference between a demo and something a brand team would use.
- It is honest about its edges. SSO, real DOCX verification and the concurrent-append race are written down with reasons, in the product and in this report, rather than papered over. That is the same discipline the product asks of its users.
- It runs. The public link works without credentials, the migrations are applied to a real database with real policies, and the whole thing is deployed on Cloudflare Workers with the parsers bundled rather than assumed.

Live site	https://site-a3e60bfeac664aac8b66164e0943ddf4.freebuff.page

Repository	https://github.com/ishmeettt/brandlens
Demo sign-in	judge@brandlens.demo · BrandLens-Judges-2026

BrandLens · brand governance with receipts · report generated from the same PDF writer the product uses for its audit export.